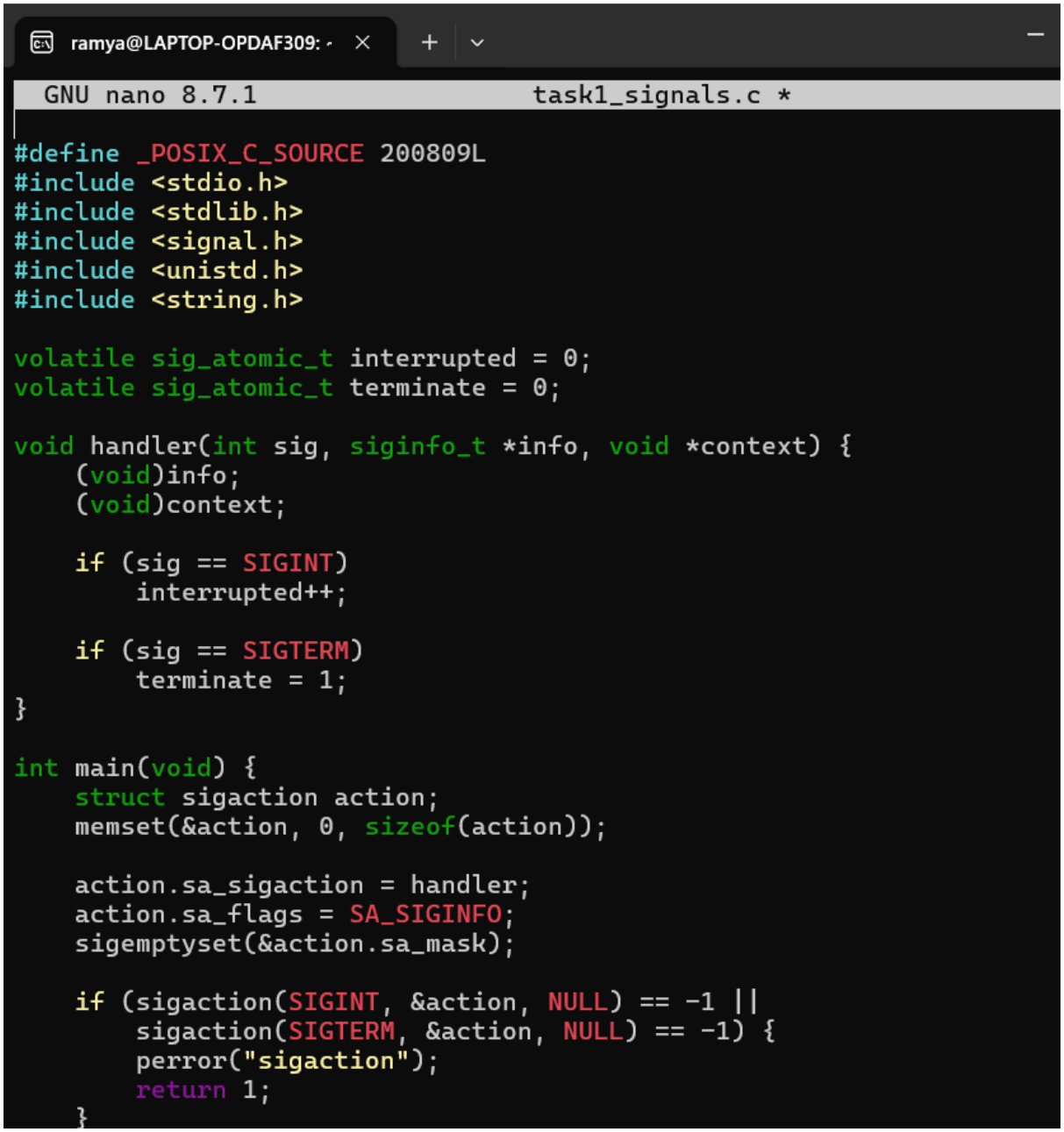
WEEK-10

Task 1 : To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events



```c
    printf("Signal handlers registered successfully.\n");
    printf("Process ID: %ld\n", (long)getpid());
    printf("Press Ctrl+C to test SIGINT.\n");
    printf("Use kill -TERM %ld to test SIGTERM.\n",
           (long)getpid());
    fflush(stdout);

    while (!terminate) {
        pause();

        if (interrupted > 0) {
            printf("SIGINT received. Program is still running.\n");
            interrupted = 0;
            fflush(stdout);
        }
    }

    printf("SIGTERM received. Exiting safely.\n");
    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/week10$ nano task1_signals.c
ramya@LAPTOP-OPDAF309:~/week10$ gcc -Wall -Wextra task1_signals.c -o task1_s
ignals
ramya@LAPTOP-OPDAF309:~/week10$ ./task1_signals
Signal handlers registered successfully.
Process ID: 642
Press Ctrl+C to test SIGINT.
Use kill -TERM 642 to test SIGTERM.
^CSIGINT received. Program is still running.
SIGTERM received. Exiting safely.
ramya@LAPTOP-OPDAF309:~/week10$ ls
task1_signals  task1_signals.c
ramya@LAPTOP-OPDAF309:~/week10$
```

Task 2 : To Capture SIGINT, Forward Signals, Protect Shell Process,Terminate Foreground Jobs, Update Job States, Test Interrupt handling

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task2_interrupt.c *

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

int main(void) {
    struct sigaction sa;
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Simple shell started. PID: %ld\n", (long)getpid());
    printf("Starting a foreground job for 30 seconds...\n");
    fflush(stdout);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }
```

```c
    if (pid == 0) {
        struct sigaction child_sa;
        child_sa.sa_handler = SIG_DFL;
        sigemptyset(&child_sa.sa_mask);
        child_sa.sa_flags = 0;

        if (sigaction(SIGINT, &child_sa, NULL) == -1) {
            perror("child sigaction");
            _exit(1);
        }

        execlp("sleep", "sleep", "30", (char *)NULL);
        perror("execlp");
        _exit(127);
    }

    int status;
    pid_t result;

    do {
        result = waitpid(pid, &status, 0);
    } while (result == -1 && errno == EINTR);

    if (result == -1) {
        perror("waitpid");
        return 1;
    }
```

```c
    if (WIFSIGNALED(status)) {
        printf("Foreground job terminated by signal %d.\n",
               WTERMSIG(status));
    } else if (WIFEXITED(status)) {
        printf("Foreground job exited with status %d.\n",
               WEXITSTATUS(status));
    }

    printf("Shell is still running and protected from SIGINT.\n");
    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/week10$ nano task2_interrupt.c
ramya@LAPTOP-OPDAF309:~/week10$ gcc -Wall -Wextra task2_interrupt.c -o task2
_interrupt
ramya@LAPTOP-OPDAF309:~/week10$ ./task2_interrupt
Simple shell started. PID: 917
Starting a foreground job for 30 seconds...
^CForeground job terminated by signal 2.
Shell is still running and protected from SIGINT.
ramya@LAPTOP-OPDAF309:~/week10$ ./task2_interrupt
Simple shell started. PID: 932
Starting a foreground job for 30 seconds...
Foreground job exited with status 0.
Shell is still running and protected from SIGINT.
ramya@LAPTOP-OPDAF309:~/week10$ ls -l task2_interrupt.c task2_interrupt
-rwxr-xr-x 1 ramya ramya 16544 Oct 12 06:39 task2_interrupt
-rw-r--r-- 1 ramya ramya  1596 Oct 12 06:39 task2_interrupt.c
ramya@LAPTOP-OPDAF309:~/week10$
```

Task 3 : To Capture SIGTSTP, Suspend Processes, Update Job Table,Preserve State, Resume Later, Test Suspension Workflow

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task3_suspension.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

volatile sig_atomic_t suspend_requested = 0;

void handle_sigtstp(int sig) {
    (void)sig;
    suspend_requested = 1;
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = handle_sigtstp;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGTSTP, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Parent process started. PID: %ld\n", (long)getpid());
    printf("Press Ctrl+Z to suspend the child process.\n");
    fflush(stdout);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }
```

```c
    if (pid == 0) {
        struct sigaction child_sa;
        child_sa.sa_handler = SIG_DFL;
        sigemptyset(&child_sa.sa_mask);
        child_sa.sa_flags = 0;

        if (sigaction(SIGTSTP, &child_sa, NULL) == -1) {
            perror("child sigaction");
            _exit(1);
        }

        while (1) {
            printf("Child process running. PID: %ld\n",
                   (long)getpid());
            fflush(stdout);
            sleep(2);
        }
    }
```

```c
    printf("Child process created. PID: %ld\n", (long)pid);
    fflush(stdout);

    while (1) {
        int status;
        pid_t result;

        do {
            result = waitpid(pid, &status, WUNTRACED);
        } while (result == -1 && errno == EINTR);

        if (result == -1) {
            perror("waitpid");
            return 1;
        }

        if (WIFSTOPPED(status)) {
            printf("\nChild suspended by signal %d.\n",
                   WSTOPSIG(status));
            printf("Job table: child PID %ld is STOPPED.\n",
                   (long)pid);
            printf("Press Enter to resume the child...\n");
            fflush(stdout);

            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
            }

            if (kill(pid, SIGCONT) == -1) {
                perror("SIGCONT");
                return 1;
            }
```

```
            printf("Job table: child PID %ld is RUNNING again.\n",
                   (long)pid);
            fflush(stdout);
        } else if (WIFEXITED(status) || WIFSIGNALED(status)) {
            printf("Child process has terminated.\n");
            break;
        }
    }

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/week10$ nano task3_suspension.c
ramya@LAPTOP-OPDAF309:~/week10$ gcc -Wall -Wextra task3_suspension.c -o task
3_suspension
ramya@LAPTOP-OPDAF309:~/week10$ ./task3_suspension
Parent process started. PID: 998
Press Ctrl+Z to suspend the child process.
Child process created. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
^Z
Child suspended by signal 20.
Job table: child PID 999 is STOPPED.
Press Enter to resume the child...

Job table: child PID 999 is RUNNING again.
Child process running. PID: 999
```

```
Job table: child PID 999 is RUNNING again.
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
Child process running. PID: 999
^C
ramya@LAPTOP-OPDAF309:~/week10$ ls -l task3_suspension.c task3_suspension
-rwxr-xr-x 1 ramya ramya 16712 Oct 12 06:43 task3_suspension
-rw-r--r-- 1 ramya ramya  2415 Oct 12 06:43 task3_suspension.c
ramya@LAPTOP-OPDAF309:~/week10$
```

Task 4 : To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

int main(void) {
    if (!isatty(STDIN_FILENO)) {
        fprintf(stderr, "Please run this program in an interactive terminal>
        return 1;
    }

    pid_t shell_group = getpgrp();
    pid_t terminal_group = tcgetpgrp(STDIN_FILENO);

    if (terminal_group == -1) {
        perror("tcgetpgrp");
        return 1;
    }

    if (terminal_group != shell_group) {
        fprintf(stderr, "This program must run in the foreground.\n");
        return 1;
    }
```

```c
    struct sigaction ignore_action;
    ignore_action.sa_handler = SIG_IGN;
    sigemptyset(&ignore_action.sa_mask);
    ignore_action.sa_flags = 0;

    int signals[] = {SIGINT, SIGQUIT, SIGTSTP, SIGTTOU, SIGTTIN};
    for (size_t i = 0; i < sizeof(signals) / sizeof(signals[0]); i++) {
        if (sigaction(signals[i], &ignore_action, NULL) == -1) {
            perror("sigaction");
            return 1;
        }
    }

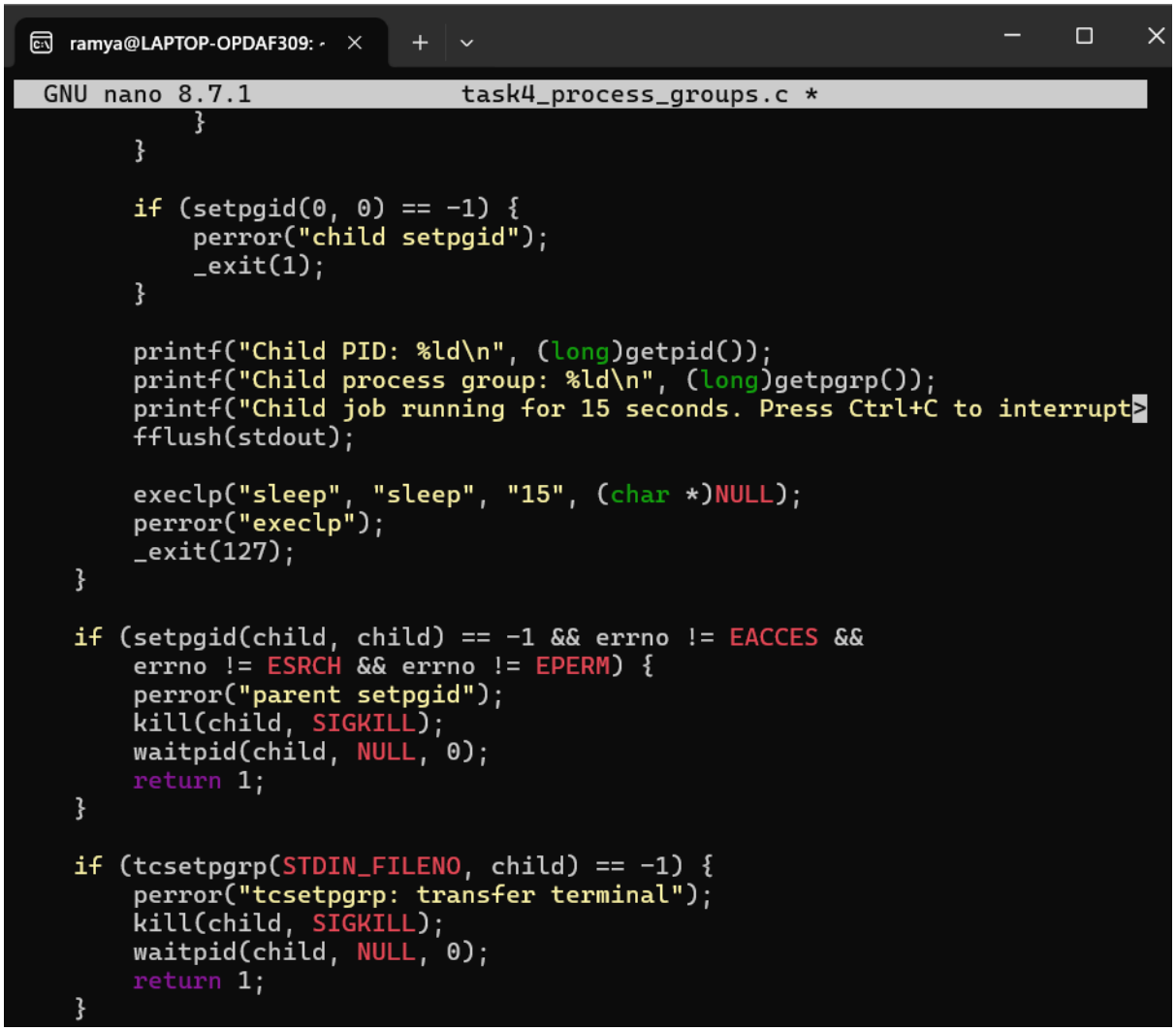
    printf("Parent PID: %ld\n", (long)getpid());
    printf("Parent process group: %ld\n", (long)shell_group);
    printf("Original terminal foreground group: %ld\n",
           (long)terminal_group);
    fflush(stdout);

    pid_t child = fork();

    if (child == -1) {
        perror("fork");
        return 1;
    }

    if (child == 0) {
        struct sigaction default_action;
        default_action.sa_handler = SIG_DFL;
        sigemptyset(&default_action.sa_mask);
        default_action.sa_flags = 0;

        for (size_t i = 0; i < sizeof(signals) / sizeof(signals[0]); i++) {
            if (sigaction(signals[i], &default_action, NULL) == -1) {
                perror("child sigaction");
                _exit(1);
```


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task4_process_groups.c *
}
}
if (setpgid(0, 0) == -1) {
perror("child setpgid");
_exit(1);
}
printf("Child PID: %ld\n", (long)getpid());
printf("Child process group: %ld\n", (long)getpgrp());
printf("Child job running for 15 seconds. Press Ctrl+C to interrupt>
fflush(stdout);
execlp("sleep", "sleep", "15", (char *)NULL);
perror("execlp");
_exit(127);
}
if (setpgid(child, child) == -1 && errno != EACCES &&
errno != ESRCH && errno != EPERM) {
perror("parent setpgid");
kill(child, SIGKILL);
waitpid(child, NULL, 0);
return 1;
}
if (tcsetpgrp(STDIN_FILENO, child) == -1) {
perror("tcsetpgrp: transfer terminal");
kill(child, SIGKILL);
waitpid(child, NULL, 0);
return 1;
}

```c
    printf("Terminal control transferred to child group %ld.\n",
           (long)child);
    fflush(stdout);

    int status;
    pid_t result;

    do {
        result = waitpid(child, &status, WUNTRACED);
    } while (result == -1 && errno == EINTR);

    if (result == -1) {
        perror("waitpid");
    } else if (WIFSTOPPED(status)) {
        printf("Child stopped by signal %d.\n", WSTOPSIG(status));
        kill(-child, SIGCONT);
        kill(-child, SIGTERM);
        while (waitpid(child, &status, 0) == -1 && errno == EINTR) {
        }
    } else if (WIFSIGNALED(status)) {
        printf("Child terminated by signal %d.\n", WTERMSIG(status));
    } else if (WIFEXITED(status)) {
        printf("Child exited with status %d.\n", WEXITSTATUS(status));
    }

    if (tcsetpgrp(STDIN_FILENO, shell_group) == -1) {
        perror("tcsetpgrp: restore terminal");
        return 1;
    }

    printf("Terminal control restored to parent group %ld.\n",
           (long)shell_group);
    printf("Process-group test completed.\n");
    return result == -1 ? 1 : 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/week10$ nano task4_process_groups.c
ramya@LAPTOP-OPDAF309:~/week10$ gcc -Wall -Wextra task4_process_groups.c -o
task4_process_groups
ramya@LAPTOP-OPDAF309:~/week10$ ./task4_process_groups
Parent PID: 1075
Parent process group: 1075
Original terminal foreground group: 1075
Terminal control transferred to child group 1076.
Child PID: 1076
Child process group: 1076
Child job running for 15 seconds. Press Ctrl+C to interrupt.
^CChild terminated by signal 2.
Terminal control restored to parent group 1075.
Process-group test completed.
ramya@LAPTOP-OPDAF309:~/week10$ ./task4_process_groups
Parent PID: 1088
Parent process group: 1088
Original terminal foreground group: 1088
Terminal control transferred to child group 1089.
Child PID: 1089
Child process group: 1089
Child job running for 15 seconds. Press Ctrl+C to interrupt.
Child exited with status 0.
Terminal control restored to parent group 1088.
Process-group test completed.
ramya@LAPTOP-OPDAF309:~/week10$ ls -l task4_process_groups.c task4_process_g
roups
-rwxr-xr-x 1 ramya ramya 16880 Oct 12 06:48 task4_process_groups
-rw-r--r-- 1 ramya ramya  3740 Oct 12 06:48 task4_process_groups.c
ramya@LAPTOP-OPDAF309:~/week10$ ps -o pid,pgid,stat,cmd -p $$
    PID    PGID STAT CMD
    343     343 Ss   -bash
ramya@LAPTOP-OPDAF309:~/week10$
```